Assignment 1: Git & GitHub Fundamentals

Report

- ***Full Name:*** Jalaleddine ait el haj
- ***Student ID:*** 149557
- ***Course Code & Section:*** CSC 3324 03
- ***GitHub Username:*** 8lhaj

1. Repository Setup

I copied the starter project into `Downloads/git-session1-8lhaj`, opened it in VS Code and used Git Bash on Windows. `git init` turned the folder into a repository on `main`.

In `.gitignore` I wrote:

- `local.env`, so local config never gets uploaded
- `*.log`, which hides `app.log` and `debug.log`
- `!important.log`, which brings that one file back so Git can track it

`git status --ignored` proves it works: the three files are under "Ignored files", and `important.log` is with the untracked files.

```
jalal@spleidz MINGW64 ~/Downloads/git-session1-8lhaj (main)
$ git status --ignored
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        app.js
        assets/
        important.log
        index.html
        notes.txt
        style.css

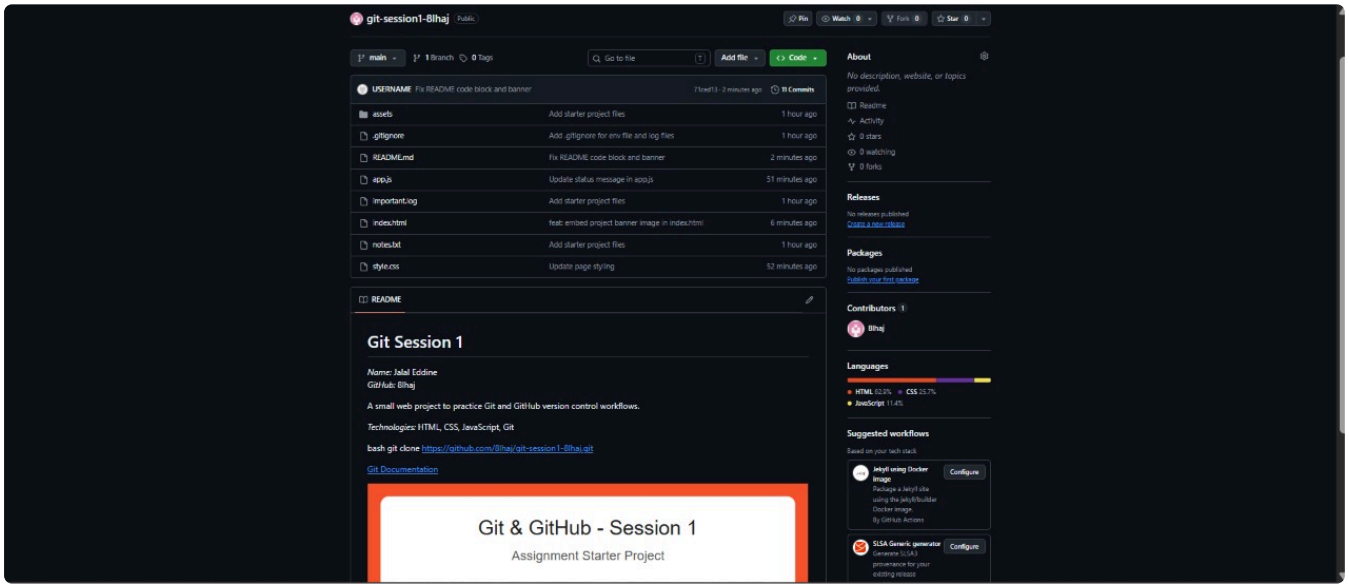
Ignored files:
  (use "git add -f <file>..." to include in what will be committed)
        app.log
        debug.log
        local.env

nothing added to commit but untracked files present (use "git add" to track)
```

2. README & Commit History

My README lists my name, my GitHub username, a one-line description, the technologies, a `git clone` code block, a link to the Git documentation and the starter banner image.

At first the banner did not show on GitHub. The code block in my README was never closed with three backticks, so everything after it was treated as code, and I had also pasted the clone URL as a Markdown link. I closed the code block, used a plain URL, put the image line `![Banner](assets/project-banner.svg)` at the end, and pushed the fix as "Fix README code block and banner". The banner now displays:



I made one commit per step:

Commit	Message
3f39d5e	Add .gitignore for env file and log files
74e6d52	Add starter project files
2b65e99	Add README with project info and banner
fc74842	Update page styling
08b3ad5	Update status message in app.js
5d9d24d	Add About section (on `feature/about`)
7ee5165	Merge feature/about into main
c5914fb	Update README with remote synchronization note (made on GitHub)

After that I added the README and banner fixes, for 11 commits on GitHub in total.

3. Selective Staging

I changed the page background in `style.css` (`#f7f7f7` to `#eef2f5`) and the status message in `app.js`. `git diff` showed the unstaged changes:

```
jalal@spleidz MINGW64 ~/Downloads/git-session1-8lhaj (main)
$ git diff
warning: in the working copy of 'app.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'style.css', LF will be replaced by CRLF the next time Git touches it
diff --git a/app.js b/app.js
index 603d28a..ac7329c 100644
--- a/app.js
+++ b/app.js
@@ -1,5 +1,5 @@
 const statusMessage = document.getElementById("status-message");

 if (statusMessage) {

jalal@spleidz MINGW64 ~/Downloads/git-session1-8lhaj (main)
$
```

Then I staged only the CSS with `git add style.css`, and `git diff --staged` showed just that file:

```
jalal@spleidz MINGW64 ~/Downloads/git-session1-8lhaj (main)
$ git add style.css
warning: in the working copy of 'style.css', LF will be replaced by CRLF the next time Git touches it

jalal@spleidz MINGW64 ~/Downloads/git-session1-8lhaj (main)
$ git diff --staged
diff --git a/style.css b/style.css
index b93e076..a0f087f 100644
--- a/style.css
+++ b/style.css
@@ -1,7 +1,7 @@
 body {
   font-family: Arial, sans-serif;
   margin: 0;
-  background: #f7f7f7;
+  background: #eef2f5;

jalal@spleidz MINGW64 ~/Downloads/git-session1-8lhaj (main)
$
```

`git status` confirms the split: `style.css` is ready to commit, `app.js` is still modified but not staged. I committed the CSS first ("Update page styling") and committed `app.js` separately right after.

```
jalal@spleidz MINGW64 ~/Downloads/git-session1-8lhaj (main)
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   style.css

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.js
```

4. Branch & Merge

I created `feature/about` with `git switch -c feature/about`, added an About section to `index.html` and committed it. Back on `main` with `git switch main`, I merged with `git merge --no-ff feature/about`. Because of `--no-ff`, Git made a separate merge commit (7ee5165), so the graph clearly shows the branch and where it joined `main`.

```
jalal@spleidz MINGW64 ~/Downloads/git-session1-8lhaj (main)
$ git log --oneline --graph --all
*   7ee5165 (HEAD -> main) Merge feature/about into main
|\
| * 5d9d24d (feature/about) Add About section
|/
* 08b3ad5 Update status message in app.js
* fc74842 Update page styling
* 2b65e99 Add README with project info and banner
* 74e6d52 Add starter project files
* 3f39d5e Add .gitignore for env file and log files
```

5. Remote Changes

I made a new empty public repository on GitHub, connected it with `git remote add origin`, and sent my work with `git push -u origin main`.

- **Commit vs push:** commit stores a snapshot on my own computer. Push copies my commits to GitHub so others can see them.
- **Fetch vs pull:** fetch only downloads what is new on GitHub into `origin/main`. My files stay the same. Pull downloads and merges, so my files change.

I tested this by editing the README on GitHub. After `git fetch`, `git status` said my branch was behind `origin/main` by 1 commit and could be fast-forwarded, while my working tree was still clean:

```
jalal@spleidz MINGW64 ~/Downloads/git-session1-8lhaj (main)
$ git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)

nothing to commit, working tree clean
```

`git pull` then fast-forwarded `main` to c5914fb, and `main` and `origin/main` point to the same commit:

```
jalal@spleidz MINGW64 ~/Downloads/git-session1-8lhaj (main)
$ git pull
Updating 7ee5165..c5914fb
Fast-forward
 README.md | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)

jalal@spleidz MINGW64 ~/Downloads/git-session1-8lhaj (main)
$ git log --oneline --graph --all
* c5914fb (HEAD -> main, origin/main) Update README with remote synchronization note
*   7ee5165 Merge feature/about into main
|\
| * 5d9d24d (feature/about) Add About section
|/
* 08b3ad5 Update status message in app.js
* fc74842 Update page styling
* 2b65e99 Add README with project info and banner
* 74e6d52 Add starter project files
* 3f39d5e Add .gitignore for env file and log files
```

6. Fair Use of AI

I followed a step-by-step guide that was prepared with an AI assistant, and I used AI again when my banner was not showing on GitHub. It pointed out that my README code block was not closed. I typed all the commands and made the file edits myself, and I verified each step with `git status`, `git diff` and `git log --oneline --graph --all` before moving on. AI also helped me write this report from my screenshots, and I checked it against what I actually did.

7. Sources & Reflection

Sources

- Session 1 slides and practice files
- Git documentation

Problems I ran into

- **Banner not showing on GitHub:** an unclosed code block in `README.md` swallowed the rest of the file. Fixed by closing it with three backticks and using a plain clone URL.
- **"LF will be replaced by CRLF" warnings:** Windows and Git use different line endings. This is only a warning, so the commits still worked.
- **`git diff` pager:** long output opens a viewer ending with `:`. Pressing `q` exits.

Reflection

The hardest concept for me was the staging area: understanding that `git add` only prepares a change, and that I can choose which files go into the next commit. The README problem also taught me that Git can work perfectly while the result on GitHub still looks wrong, so I now check the GitHub page after every push. If Git behaves unexpectedly, the first thing I would check is `git status`, because it shows my branch, what is staged and whether I am ahead of or behind GitHub.